

Deliverable 2 PermitNow

Simone Battisti 242708 - Davide Basso 244822 - Nicola Avellino 243294

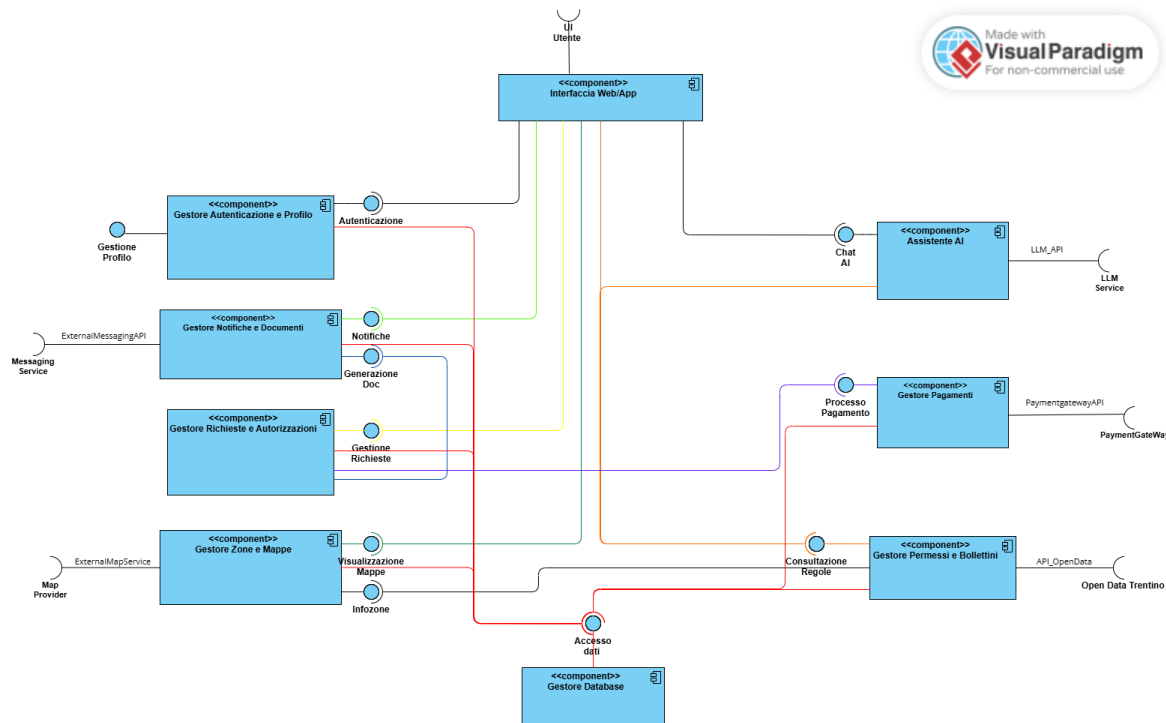
Contents

1	Diagramma Componenti	2
1.1	Diagramma dei componenti	2
1.2	Descrizione dei componenti	2
1.3	Motivazione della suddivisione architetturale	4
2	Diagramma delle Classi	5
2.1	User - UserManager	5
2.2	FishingLicense - FishingLicensesManager	5
2.3	FishingPermit - FishingPermitsManager	6
2.4	HuntingLicense - HuntinLicenseManager	7
2.5	HuntingPage - HuntingPagesManager	7
2.6	LocalPermit - LocalPermitsManager	8
2.7	Diagramma	9
3	Constraint OCL	9
3.1	Vincoli di Integrità: Entità User	9
3.2	Vincoli Operativi: FishingPermit e LocalPermit	10
3.3	Vincoli del Libretto Venatorio: HuntingPage	10
3.4	Vincoli di Stato: Licenze	11
3.5	Contratti dei Manager	11

1 Diagramma Componenti

In questa sezione viene riportata una prima progettazione architettuale basata sul diagramma dei componenti del sistema da realizzare, cercando di minimizzare l'accoppiamento (connessioni tra componenti diverse) e massimizzare la coesione (connessioni interne al componente). Riportiamo di seguito il diagramma delle componenti e, per ogni componente, una breve descrizione del suo scopo e delle sue interfacce (classificandole in fornite e richieste).

1.1 Diagramma dei componenti



1.2 Descrizione dei componenti

Interfaccia Web/App

Rappresenta il front-end del sistema PermitNow. È il componente che media l'interazione tra l'utente (cittadino o amministratore) e la logica di business. Non gestisce direttamente i dati, ma delega ogni operazione ai gestori di competenza tramite le loro interfacce.

Tipologia	Nome	Descrizione
Fornita	UI Utente	Fornisce l'interfaccia grafica per l'accesso alle funzionalità (Web/Mobile).
Richiesta	Autenticazione	Richiesta al <i>Gestore Autenticazione</i> per login e gestione sessione.
Richiesta	ConsultazioneRegole	Richiesta al <i>Gestore Permessi</i> per la ricerca di normative e bollettini.
Richiesta	GestioneRichieste	Richiesta al <i>Gestore Richieste</i> per l'invio di pratiche e monitoraggio stato.
Richiesta	VisualizzazioneMappe	Richiesta al <i>Gestore Zone e Mappe</i> per la consultazione cartografica.
Richiesta	ChatAI	Richiesta all' <i>Assistente AI</i> per il supporto conversazionale.
Richiesta	Notifiche	Richiesta al <i>Gestore Notifiche</i> per la ricezione di avvisi e documenti.

Gestore Autenticazione e Profilo

Componente critico per la sicurezza. Gestisce l'identità digitale, la persistenza dei profili e la logica dei ruoli (RBAC - Role Based Access Control), garantendo che solo gli amministratori accedano alle funzioni gestionali.

Tipologia	Nome	Descrizione
Fornita	Autenticazione	Fornisce i servizi di registrazione, login, logout e validazione token.
Fornita	GestioneProfilo	Espone i metodi per la lettura e modifica dei dati anagrafici dell'utente.
Richiesta	AccessoDati	Utilizza l'interfaccia del <i>Gestore Database</i> per persistere le credenziali e i profili.

Gestore Permessi e Bollettini

È il “motore normativo” del sistema. Aggrega i dati locali con i flussi informativi esterni per fornire una visione aggiornata su cosa sia consentito fare, dove e quando.

Tipologia	Nome	Descrizione
Fornita	ConsultazioneRegole	Espone le logiche di ricerca e filtraggio per permessi, bollettini e regolamenti.
Richiesta	AccessoDati	Richiesta al DB per recuperare la normativa salvata nel sistema.
Richiesta	APIOpenData	Interfaccia richiesta a sistemi esterni (es. PA/Open Data Trentino) per dati live.
Richiesta	InfoZone	Utilizza il <i>Gestore Zone e Mappe</i> per verificare i vincoli geografici legati a un permesso.

Gestore Richieste e Autorizzazioni

Gestisce il workflow transazionale del sistema. Si occupa della creazione di una richiesta, del passaggio di stato (es. “in attesa”, “approvato”) e del collegamento con il modulo pagamenti.

Tipologia	Nome	Descrizione
Fornita	GestioneRichieste	Fornisce i metodi per sottoporre una pratica e seguirne l'iter burocratico.
Richiesta	AccessoDati	Utilizza il DB per salvare lo stato delle richieste e i documenti allegati.
Richiesta	ProcessoPagamento	Si appoggia al <i>Gestore Pagamenti</i> per le transazioni economiche.
Richiesta	GenerazioneDoc	Richiede al <i>Gestore Notifiche</i> la creazione del PDF finale (permesso/ricevuta).

Gestore Zone e Mappe

Componente specializzato nella logica GIS (Geographic Information System). Traduce le coordinate e i poligoni del database in layer visualizzabili e interroga le aree per verificare la validità territoriale delle attività.

Tipologia	Nome	Descrizione
Fornita	VisualizzazioneMappe	Fornisce i layer grafici da visualizzare nell'interfaccia utente.
Fornita	InfoZone	Espone servizi di verifica spaziale (es. “questa coordinata è in zona protetta?”).
Richiesta	AccessoDati	Recupera i dati geometrici e i vincoli delle zone dal database.
Richiesta	ExternalMapService	Interfaccia richiesta a provider esterni (es. Google Maps/Mapbox) per la cartografia di base.

Assistente AI

Componente di supporto avanzato. Utilizza modelli linguistici per semplificare l'esperienza utente, rendendo la burocrazia più accessibile tramite linguaggio naturale.

Tipologia	Nome	Descrizione
Fornita	ChatAI	Fornisce l'interfaccia di messaggistica intelligente.
Richiesta	ConsultazioneRegole	Fondamentale per fornire risposte basate sulle regole reali del sistema.
Richiesta	LLM_API	Interfaccia richiesta verso servizi esterni di intelligenza artificiale (es. OpenAI).

Gestore Notifiche e Documenti

Responsabile della comunicazione asincrona e della produzione documentale. Isola la logica di invio messaggi e generazione PDF dal resto del sistema.

Tipologia	Nome	Descrizione
Fornita	Notifiche	Fornisce metodi per accodare notifiche e generare documenti digitali.
Richiesta	ExternalMessagingAPI	Interfaccia richiesta a gateway esterni (Email/Push Notifications).
Richiesta	AccessoDati	Recupera i template dei documenti e i recapiti utente dal database.

Gestore Pagamenti

Componente “Wrapper” che isola il sistema dai provider finanziari. Garantisce che PermitNow non debba gestire direttamente dati sensibili (come carte di credito), aumentando la sicurezza.

Tipologia	Nome	Descrizione
Fornita	ProcessoPagamento	Espone i metodi per avviare una transazione e riceverne l'esito.
Richiesta	PaymentGatewayAPI	Interfaccia richiesta verso circuiti esterni (es. Stripe, PayPal, PagoPA).
Richiesta	AccessoDati	Registra le transazioni e gli esiti nel database di sistema.

Gestore Database

Funge da *Data Access Layer (DAL)*. È l'unico componente autorizzato a interagire con il DBMS fisico. Questa scelta architetturale garantisce che, se si decidesse di cambiare database, solo questo componente andrebbe modificato.

Tipologia	Nome	Descrizione
Fornita	AccessoDati	Fornisce un set completo di operazioni CRUD a tutti gli altri componenti.
Richiesta	DBMS_Interface	Connessione fisica al motore del database (es. PostgreSQL o MongoDB).

1.3 Motivazione della suddivisione architetturale

L'architettura di **PermitNow** è stata progettata seguendo il principio della **scomposizione modulare**, mirata a massimizzare la coesione e minimizzare l'accoppiamento.

- **Alta Coesione:** Ogni componente è specializzato. Ad esempio, il *Gestore Zone e Mappe* si occupa esclusivamente di logica geografica, mentre il *Gestore Pagamenti* isola la complessità finanziaria. Questo facilita la manutenzione e il testing isolato.
- **Basso Accoppiamento:** I componenti comunicano esclusivamente tramite interfacce ben definite. L'uso del *Gestore Database* come punto unico di accesso ai dati impedisce che modifiche alla struttura delle tabelle abbiano un impatto a catena su tutto il sistema (*Principio di Information Hiding*).
- **Sostituibilità:** Grazie all'uso delle interfacce verso l'esterno (come per i pagamenti o l'AI), è possibile sostituire un fornitore di servizi (es. passare da Google Maps a OpenStreetMap) aggiornando un solo componente senza alterare la logica complessiva della piattaforma.

2 Diagramma delle Classi

Tutte le classi Kotlin di PermitNow hanno come parametro un valore **connection**, il quale determina la connessione al Database tramite l'utilizzo di una libreria chiamata **Kotlin Exposed** per la gestione dei DB tramite codice.

Per ogni entità all'interno del servizio viene definito il corrispettivo **Manager** che conterrà tutte le funzioni per gestirla. Ogni entità è definita da un file **JSON** che permette il corretto parsing delle informazioni, tramite l'utilizzo di **Kotlin Serialization**.

Ogni manager può definire delle funzioni accessibili a tutti gli utenti ed altre utilizzabili solamente dagli admin, dato che il sistema gestisce una gestione dei diversi **Tenant** con un amministratore per ognuno. Nel nostro caso un Tenant è un'organizzazione che ha accesso ai dati, anche in modo parziale in base alle proprietà del Tenant stesso.

2.1 User - UserManager

Questa classe gestisce tutte le azioni dell'entità **Utente**, definita dalla classe *RegisterJson*, composta da:

```
1 @Serializable
2 class RegisterJson (
3     val name: String,
4     val surname: String,
5     val email: String,
6     val password: String,
7     val fiscalCode: String = "",
8     val role: String = "user",
9     val verified: Boolean = false
10 )
```

Il suo manager definisce le funzioni:

- **login(loginJson: LoginJson)**: permette il login dell'utente all'interno del sistema. Vengono controllati email e password sul sistema e viene ritornato l'ID dell'utente nel caso di autenticazione corretta. Prende come parametro un LoginJson, il quale definisce un JSON con all'interno email e password dell'utente.
- **register(userJson: UserJson)**: permette la registrazione di un utente all'interno del sistema, tramite le informazioni di un UserJson definito sopra. L'utente verrà inserito nel sistema come utente "user" non verificato. La verifica avverrà solamente con l'accesso tramite SPID successivo alla registrazione.
- **adminLogin(loginJson: LoginJson)**: login per il lato admin del sistema, utilizzato per accedere alla dashboard. Sfrutta come parametro sempre un LoginJson, ma in questo caso controlla che l'utente sia un admin nel sistema prima di garantire l'accesso.
- **getUserInfo(userId: Int)**: questa funzione permette la ricezione di un UserJson senza password (per motivi di sicurezza) a chi la richiama. Prende come parametro l'intero identificativo dell'utente.
- **updateUser(userId: Int, registerJson: RegisterJson)**: permette la modifica di un utente all'interno del sistema. Prende come parametro uno userId e il registerJson, entrambi definiti precedentemente.
- **verifyUser(userId: Int)**: permette di verificare il proprio utente dopo la verifica tramite SPID o può essere utilizzato dall'amministratore del Tenant dell'organizzazione coi massimi permessi di verificare manualmente un utente.

2.2 FishingLicense - FishingLicensesManager

In questa classe gestiamo le **licenze di pesca**, definite dalla classe *FishingLicense*:

```
1 @Serializable
2 class FishingLicenseJson(
3     val licenseId: Int,
4     val licenseNumber: String,
5     val releasedBy: String, // Associazione Rilasciante
6     val season: String, // Stagione di pesca corrente
7     val noKill: Boolean, // Parametro di kill del pescato
8     val type: String, // valid o pending
9     val qrCodeToken: String,
10     val userId: Int
11 )
```

Il suo manager definisce le funzioni:

- **getLicenceFromImage(multipart: Map<String, String>, userId: Int) → Int**: permette la creazione di una nuova licenza tramite una foto di quella fisica. Inserisce una nuova licenza all'interno del con tutti i parametri definiti dalla classe FishingLicenseJson definita sopra. Questa licenza avrà il parametro *type* con valore "pending", perché aspetta la verifica da parte dell'amministratore prima di essere verificata e passare al tipo "verified".
- **isValid(user: UserJson, licenseJson: FishingLicenseJson) → Boolean**: ritorna true se l'utente che sta richiedendo la licenza soddisfa i requisiti di sistema e che chi ha fatto il login sia il proprietario della licenza.
- **getLicenseInfo(licenseId: Int) → FishingLicenseJson**: ritorna un FishingLicenseJson con all'interno le informazioni della licenza definita dall'id *licenseId*.
- **isFishingLicense(licenseId: Int) → Boolean**: ritorna true se la licenza è una licenza di pesca presente nel sistema.
- **updateLicenseData(formData: FishingLicenseUpdateForm, licenseId: Int)**: funzione esposta solo per gli amministratori del Tenant con i permessi di gestione della classe corrente. Permette ad un amministratore di modificare le informazioni riguardo la licenza identificata da licenseId. LicenseFormData è una funzione definita da:

```

1 @Serializable
2 class FishingLicenseUpdateForm(
3     val licenseNumber: String,
4     val nokill: Boolean,
5     val releasedBy: String,
6     val season: String,
7     val type: String
8 )

```

2.3 FishingPermit - FishingPermitsManager

Questa classe definisce un permesso di pesca temporaneo definito dalla classe *FishingPermit*:

```

1 @Serializable
2 class FishingPermitJson(
3     val permitId: Int,
4     val type: String, // daily, weakly o annual
5     val expireDate: String, // Data di scadenza
6     val qrCodeToken: String,
7     val userId: Int? = null,
8     val fishingLimit: Int? = null // Limite di pescabile
9 )

```

Il suo manager definisce le funzioni:

- **createNewPermit(newPermitJson: NewFishingPermitJson) → Int**: permette di creare un nuovo permesso di pesca temporaneo. Il JSON in input è definito da:

```

1 @Serializable
2 class NewFishingPermitJson(
3     val type: String,
4     val fiscalCode: String,
5     val limit: Int,
6 )

```

- **getPermitsForUser(userId: Int) → List<FishingPermitJson>**: permette all'utente di vedere tutti i propri permessi di pesca.
- **getPermitInfo(permitId: Int) → FishingPermitJson**: permette di ottenere le informazioni riguardo ad un permesso. Il permesso viene selezionato sul sistema tramite l'id univoco passato come parametro alla funzione.
- **getValidPermits() → List<FishingPermitJson>**: funzione esposta solo per gli amministratori del Tenant con i permessi di gestione della classe corrente. Permette ad un amministratore di ottenere una lista di tutte i permessi non scaduti presenti sul sistema.
- **updatePermit(permitId: Int, permitJson: FishingPermitJson)**: funzione esposta solo per gli amministratori del Tenant con i permessi di gestione della classe corrente. Permette ad un amministratore di modificare i parametri di un permesso.

2.4 HuntingLicense - HuntinLicenseManager

Questa classe definisce le *Licenza di Caccia*. Ad ogni licenza di caccia è collegato il rispettivo *Libretto di Caccia*. Ogni licenza è definita da:

```
1 @Serializable
2 class HuntingLicenseJson(
3     val licenceId: Int,
4     val licenseNumber: String,
5     val releaseDate: String, // Data di rilascio
6     val home: String, // Residenza di caccia
7     val status: String, // valid o pending
8     val qrCodeToken: String,
9     val userId: Int
10 )
```

Il suo manager definisce le funzioni:

- **getLicenceFromImage(multipart: Map<String, String>, userId: Int) → Int**: permette la creazione di una nuova licenza tramite una foto di quella fisica. Inserisce una nuova licenza all'interno del con tutti i parametri definiti dalla classe HuntingLicenseJson definita sopra. Questa licenza avrà il parametro *type* con valore *"pending"*, perché aspetta la verifica da parte dell'amministratore prima di essere verificata e passare al tipo *"verified"*. Crea allo stesso tempo il libretto di caccia per la stagione corrente.
- **isValid(user: UserJson, licenseJson: HuntingLicenseJson) → Boolean**: ritorna *true* se l'utente che sta richiedendo la licenza soddisfa i requisiti di sistema e che chi ha fatto il login sia il proprietario della licenza.
- **getLicenseInfo(licenseId: Int) → HuntingLicenseJson**: ritorna un HuntingLicenseJson con all'interno le informazioni della licenza definita dall'id *licenseId*.
- **isFishingLicense(licenseId: Int) → Boolean**: ritorna *true* se la licenza è una licenza di pesca presente nel sistema.
- **updateLicenseData(formData: HuntingLicenseUpdateJson, licenseId: Int)**: funzione esposta solo per gli amministratori del Tenant con i permessi di gestione della classe corrente. Permette ad un amministratore di modificare le informazioni riguardo la licenza identificata da *licenseId*. LicenseFormData è una funzione definda da:

```
1 @Serializable
2 class HuntingLicenseUpdateJson(
3     val licenseNumber: String,
4     val releaseDate: String,
5     val home: String,
6     val status: String,
7 )
8
```

2.5 HuntingPage - HuntingPagesManager

Questa classe definisce una pagina del *Libretto di Caccia*, con riferimento: *1 licenza di caccia → n pagine*. Ogni pagina del libretto di caccia viene definito da:

```
1 @Serializable
2 class HuntingPageJson(
3     val id: Int,
4     val licenseId: Int,
5     val pageDate: String,
6     val place: String, // Luogo di venatura
7     val type: String, // Caccia vagante, fissa, ...
8     val outOfRegion: Boolean, // Fuori Regione
9     val animals: String, // Mappa <String, Int> contentente i risultati di venatura
10    val atc: String, // Ambito Territoriale di Caccia
11    val cac: String, // Comprensorio Alpino di Caccia
12    val afv: Boolean, // Azienda Faunistico Venatoria
13    val aatv: Boolean, // Aienda Agri Turistico Venatoria
14    val mobility: Boolean, // Mobilità venatoria
15    val outRegion: String, // Regione in cui ci si trova, se attivo il flag
    outOfRegion
```

```

16     val typeOfBirds: String, // Tipi di uccelli cacciabili: "turidi", "acquatici" o "
    alldole"
17     val userId: Int
18 )

```

Il suo manager definisce le funzioni:

- **createNewPage(huntingPageJson: *HuntingPageJson*) → *Int***: genera una nuova pagina del libretto. Prende come parametro il *HuntingPageJson* definito prima.
- **addAnimalKill(animalName: *String*, pageId: *Int*)**: aggiunge 1 al counter per un singolo animale. Prende in input il nome dell'animale cacciato e l'id univoco della pagina.
- **getAllPages(licenseId: *Int*) → *List<HuntingPageJson>***: permette all'utente di avere una lista di tutte le pagine del proprio libretto.
- **updatePage(huntinPageJson: *HuntinPageJson*)**: funzione esposta solo per gli amministratori del Tenant con i permessi di gestione della classe corrente. Permette ad un amministratore di modificare i parametri di una pagina.
- **assignNewBook() → *Int***: funzione esposta solo per gli amministratori del Tenant con i permessi di gestione della classe corrente. Permette ad un amministratore di recuperare tutti i libretti e azzerarli per la nuova stagione di caccia.

2.6 LocalPermit - LocalPermitsManager

Questa classe definisce i *Permessi Locali* per la raccolta funghi. Di default hanno un limite di 2kg a permesso e possono essere giornalieri, settimanali o stagionali. Sono definiti da:

```

1 @Serializable
2 class LocalPermitJson(
3     val permitId: Int,
4     val type: String, // daily, weakly o annual
5     val expireDate: String, // Data di scadenza
6     val qrCodeToken: String,
7     val userId: Int? = null,
8     val zone: String // Zona di raccolta
9 )

```

Il suo manager definisce le funzioni:

- **createNewPermit(newPermitJson: *NewPermitJson*) → *Int***: permette di creare un nuovo permesso di pesca temporaneo. Il JSON in input è definito da:

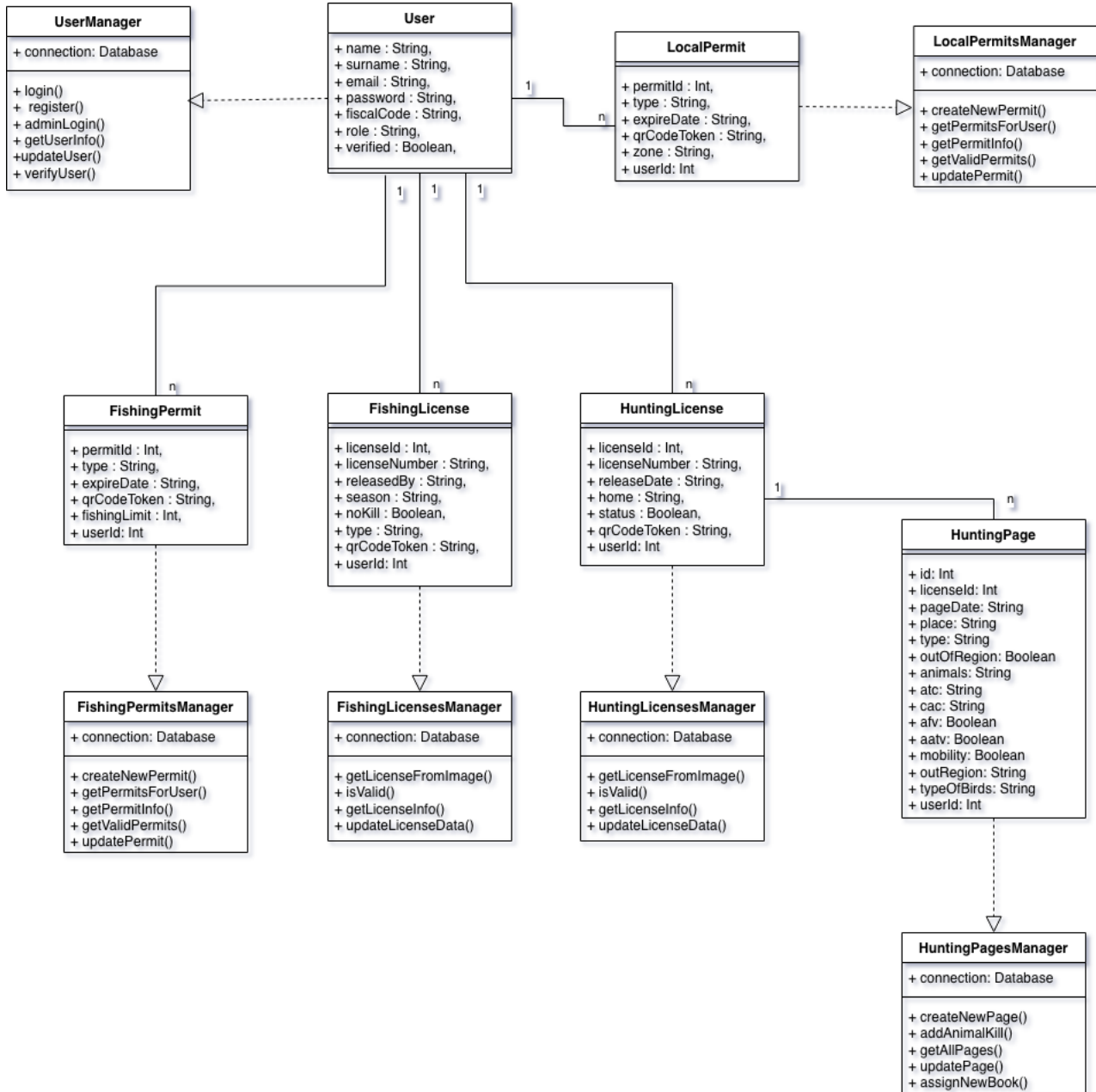
```

1 @Serializable
2 class NewLocalPermitJson(
3     val type: String,
4     val zone: String,
5     val fiscalCode: String,
6 )

```

- **getPermitsForUser(userId: *Int*) → *List<LocalPermitJson>***: permette all'utente di vedere tutti i propri permessi di raccolta funghi per una specifica zona.
- **getPermitInfo(permitId: *Int*) → *LocalPermitJson***: permette di ottenere le informazioni riguardo ad un permesso. Il permesso viene selezionato sul sistema tramite l'id univoco passato come parametro alla funzione.
- **getValidPermits() → *List<LocalPermitJson>***: funzione esposta solo per gli amministratori del Tenant con i permessi di gestione della classe corrente. Permette ad un amministratore di ottenere una lista di tutte i permessi non scaduti presenti sul sistema.
- **updatePermit(permitId: *Int*, permitJson: *LocalPermitJson*)**: funzione esposta solo per gli amministratori del Tenant con i permessi di gestione della classe corrente. Permette ad un amministratore di modificare i parametri di un permesso.

2.7 Diagramma



3 Constraint OCL

L'integrità dei dati e la coerenza delle operazioni all'interno di **PermitNow** vengono garantite attraverso l'applicazione di vincoli formali definiti in **OCL** (Object Constraint Language). Questi vincoli agiscono come uno strato di validazione logica che estende le capacità del diagramma delle classi, assicurando che lo stato del sistema rispetti le normative vigenti e le regole di business del dominio.

3.1 Vincoli di Integrità: Entità User

La classe **User** rappresenta il perno centrale del sistema. I vincoli definiti assicurano che ogni account sia univoco, correttamente tipizzato e che i titoli autorizzativi (licenze) siano gestiti secondo il principio di esclusività temporale.

- **ValidRole**: garantisce che il ruolo assegnato sia limitato ai tipi previsti dal sistema (*user* o *admin*).
- **VerifiedRequiresFiscalCode**: assicura che un utente verificato tramite SPID abbia obbligatoriamente un codice fiscale registrato.
- **UniqueEmail**: impone l'univocità dell'indirizzo email a livello di intero sistema per prevenire duplicazioni di account.
- **MaxActiveLicenses**: definisce il limite legale per cui un utente non può possedere più di una licenza valida o in fase di approvazione per la stessa categoria (Caccia o Pesca).

```

1 context User
2
3 inv ValidRole:
4 self.role = 'user' or self.role = 'admin'
5
6 inv VerifiedRequiresFiscalCode:
7 self.verified = true implies self.fiscalCode <> ''
8
9 inv FiscalCodeLength:
10 self.fiscalCode <> '' implies self.fiscalCode.size() = 16
11
12 inv UniqueEmail:
13 User.allInstances()->isUnique(email)
14
15 inv MaxOneActiveFishingLicense:
16 self.fishingLicense->select(1 | l.type = 'valid' or l.type = 'pending')->size() <= 1
17
18 inv MaxOneActiveHuntingLicense:
19 self.huntingLicense->select(1 | l.status = 'valid' or l.status = 'pending')->size()
    <= 1

```

3.2 Vincoli Operativi: FishingPermit e LocalPermit

Queste classi gestiscono i permessi temporanei di breve durata. La logica OCL qui applicata serve a prevenire l'emissione di titoli non validi o l'acquisto di permessi senza i prerequisiti legali necessari.

- **RequiresValidLicense:** stabilisce una dipendenza gerarchica per cui un permesso di pesca può essere creato solo se l'utente possiede già una licenza di pesca in stato *valid*.
- **ExpireDateValid:** garantisce che la data di scadenza di un nuovo permesso sia sempre successiva alla data odierna, impedendo l'inserimento di titoli già scaduti.
- **MandatoryData:** assicura che campi critici come il *qrCodeToken* e la zona di raccolta funghi non siano mai lasciati vuoti.

```

1 context FishingPermit
2 inv ValidPermitType:
3 self.type = 'daily' or self.type = 'weekly' or self.type = 'annual'
4
5 inv RequiresValidLicense:
6 self.user.fishingLicense->exists(1 | l.type = 'valid')
7
8 inv ValidFishingLimit:
9 self.fishingLimit->notEmpty() implies self.fishingLimit > 0
10
11 context LocalPermit
12 inv ValidPermitType:
13 self.type = 'daily' or self.type = 'weekly' or self.type = 'annual'
14
15 inv ExpireDateValid:
16 self.expireDate > today
17
18 inv ZoneNotEmpty:
19 self.zone <> ''

```

3.3 Vincoli del Libretto Venatorio: HuntingPage

La classe **HuntingPage** modella le singole giornate di attività registrate nel libretto. Data la complessità burocratica, i vincoli qui definiti prevengono paradossi logici tra la licenza del cacciatore e la pagina compilata.

- **ConsistencyUser:** vincolo di triangolazione che assicura che l'utente che compila la pagina sia lo stesso proprietario della licenza di caccia associata.
- **RegionLogic:** gestisce la coerenza dei dati geografici; se viene attivato il flag di uscita regione (*outOfRegion*), il sistema obbliga l'inserimento del nome della regione di destinazione, e viceversa.
- **LegalHuntingTypes:** limita le stringhe accettate per le tipologie di caccia e selvaggina a quelle previste dalla normativa vigente.

```

1 context HuntingPage
2
3 inv ConsistencyUser:
4 self.user = self.huntingLicense.user
5
6 inv RegionRequiredWhenOut:
7 self.outOfRegion = true implies self.outRegion <> ''
8
9 inv RegionEmptyWhenIn:
10 self.outOfRegion = false implies self.outRegion = ''
11
12 inv ValidHuntingType:
13 self.type = 'vagante' or self.type = 'appostamento_fisso' or
14 self.type = 'appostamento_temporaneo' or self.type = 'selezione'

```

3.4 Vincoli di Stato: Licenze

Le classi **FishingLicense** e **HuntingLicense** fungono da titoli autorizzativi ufficiali. I vincoli si focalizzano sulla validità temporale del documento e sulla completezza delle informazioni rilasciate dall'autorità.

- **ValidLicenseState**: limita lo stato della licenza ai soli valori ammessi (*valid*, *pending*).
- **ReleaseDateValid**: assicura l'integrità storica impedendo l'inserimento di date di rilascio future rispetto al momento della registrazione.
- **MandatoryIdentification**: impone la presenza del numero di licenza e dell'ente rilasciante per rendere il documento legalmente valido.

```

1 context FishingLicense
2 inv ValidLicenseState:
3 self.type = 'valid' or self.type = 'pending'
4
5 inv MandatoryIdentificationData:
6 self.licenseNumber <> '' and self.releasedBy <> ''
7
8 context HuntingLicense
9 inv ValidLicenseState:
10 self.status = 'valid' or self.status = 'pending'
11
12 inv ReleaseDateValid:
13 self.releaseDate <= today

```

3.5 Contratti dei Manager

Vengono infine definiti i contratti per le funzioni critiche dei **Manager**, specificando le condizioni necessarie per l'invocazione delle operazioni e lo stato garantito al termine delle stesse.

- **Pre-condizioni**: l'utente deve essere censito correttamente nel sistema prima di poter generare nuovi titoli o permessi.
- **Post-condizioni**: garantisce l'effettivo incremento della persistenza dei dati e la corretta generazione degli identificativi univoci.

```

1 context LocalPermitsManager::createNewPermit(newPermitJson: NewLocalPermitJson) : Int
2
3 pre UserMustExist:
4 User.allInstances()->exists(u | u.fiscalCode = newPermitJson.fiscalCode)
5
6 post PermitCreated:
7 LocalPermit.allInstances()->size() = LocalPermit.allInstances()@pre->size() + 1

```